ISEN
ALL IS DIGITAL!
OUEST

yncréa

Electronics Laboratory Session

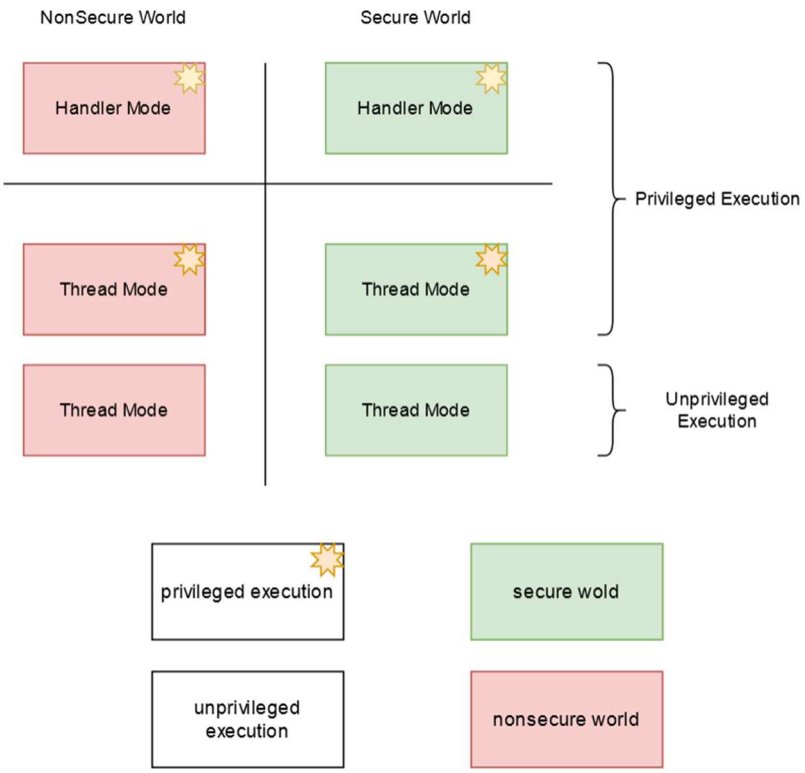
Microcontrollers Security

Trust Zone

Jean-Jacques Meneu

TrustZone, or **Security Extensions**, are an optional feature on some Armv8-M cores (Cortex M33 or Cortex M23).

Security Extensions introduce an additional partitioning of the system - the **security level** - which is orthogonal to the execution mode already available (see Chapter 3.4): **secure** world (**S**) (or **TrustZone**) and **non-secure** (**NS**) world. In both of these worlds, thread and handler mode are available. Additionally, you can assign Peripherals, Flash and RAM to one world. Once TrustZone is enabled on a processor, it always starts in S world.



Note: in handler mode (after entering in an interrupt) the processor is always in privileged mode. In other words, it can access all the internal registers. In thread mode, it has limited access to internal functions of the code (like the NVIC, system timer, stack pointers, PRIMASK…). The programming manual summarizes as written below:

Processor mode and privilege levels for software execution

The processor *modes* are:

Thread mode: Used to execute application software.
The processor enters Thread mode when it comes out of reset.
The CONTROL register controls whether software execution is privileged or unprivileged, see *CONTROL register on page 25*.

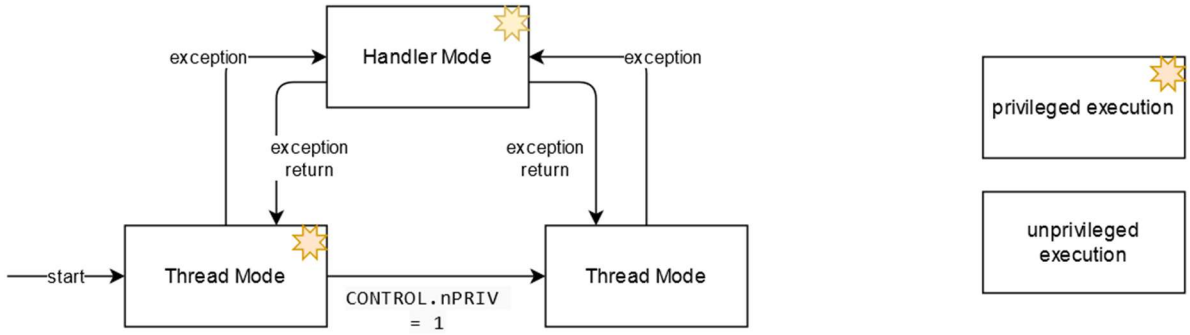
Handler mode: Used to handle exceptions.
The processor returns to Thread mode when it has finished exception processing.
Software execution is always privileged.

The *privilege levels* for software execution are:

Unprivileged: *Unprivileged software* executes at the unprivileged level and:

- Has limited access to the MSR and MRS instructions, and cannot use the CPS instruction.
- Cannot access the system timer, NVIC, or system control block.
- Might have restricted access to memory or peripherals.
- Must use the SVC instruction to make a *supervisor call* to transfer control to privileged software.

Privileged: *Privileged software* executes at the privileged level and can use all the instructions and has access to all resources.
Can write to the CONTROL register to change the privilege level for software execution.



After partitioning the memory between secure and non-secure world, non-secure world can only access non-secure memories.

Note: In cortex A, the trust zone has a so-called Security Monitor, which is the sole entry point into secure world. In Cortex M, the trust zone can have many entry points, which are placed in dedicated regions, called non-secure callable region.

Memory areas and other critical resources are marked as secure and then can only be accessed when the core is executing in secure state. The program address, the address of the instruction currently executed, determines the security state of the processor.

Memory regions are marked by a secure attribute, determining their security attribution. When TrustZone-M is enabled, the processor always starts in secure world and all memory addresses (Flash, SRAM, Peripherals) are secure.

There are three security attributes a memory region can have. One for each security level (secure and non-secure) and an additional (non-secure callable). In documentation, they are referred as:

- Non-secure (NS)
- Non-secure Callable (NSC)
- Secure (S)

Jean-Jacques Meneu

When the processor executes an instruction in an NS region, the processor is in security level non-secure.

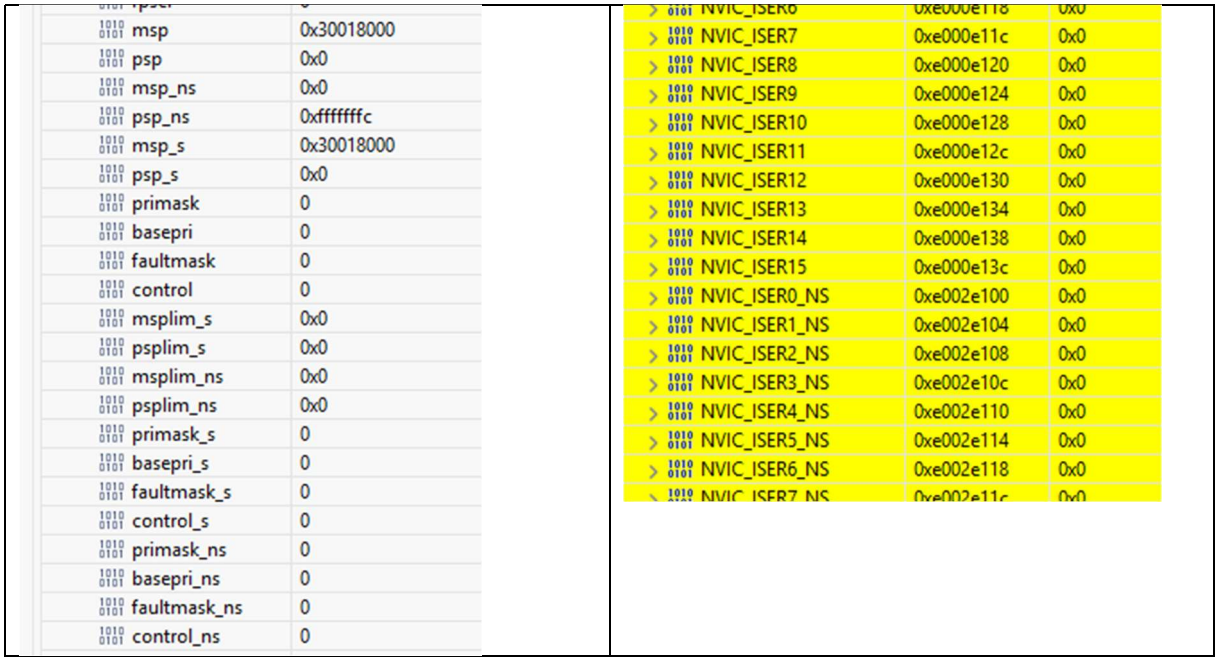
Secure world can't be called directly from Non-Secure world. To call functions in Secure world, a small entry point, which resides in a non-secure callable region, must be called. In this region, the transition to secure world takes place. For example, some registers are banked between security states. After the core has transitioned into secure state, it branches off to the actual implementation of the function being called.

Between security states, the following registers are banked. Banking a register means that there are distinct instances of these registers available. These instances are switched automatically by the core during transitioning from Non-Secure to Secure and back. The following registers are banked in ARMV8-M

- R13 : SP (Stack Pointer)
- PRIMASK, FAULTMASK, BASEPRI
- Some bits in CONTROL

The system Control Space SCS is also bankcd.

Below some examples of banked registers



IDAU and SAU: Security attribution of memory addresses is done in so-called Attribution Units

- **Security Attribution Unit (SAU):** always available in ARMv8-M Cores. On reset, SAU is disabled and con be configured by the secure application.
- **Implementation Defined Attribution Unit (IDAU):** external to the core and the presence depends on the vendors implementation.

Jean-Jacques Meneu

IDAU is static and cannot be modified for a given product. For the STM32L552, the datasheet or the reference manual gives the information. The figure below shows the addresses for each region, that can be either Non-secure or Non-secure Callable.

Region description	Address range	IDAU security attribution
Code - external memories	0x0000_0000 0x07FF_FFFF	Non-secure
Code - Flash and SRAM	0x0800_0000 0x0BFF_FFFF	Non-secure
	0x0C00_0000 0x0FFF_FFFF	NSC
Code - external memories	0x1000_0000 0x17FF_FFFF	Non-secure
	0x1800_0000 0x1FFF_FFFF	
SRAM	0x2000_0000 0x2FFF_FFFF	Non-secure
	0x3000_0000 0x3FFF_FFFF	NSC
Peripherals	0x4000_0000 0x4FFF_FFFF	Non-secure
	0x5000_0000 0x5FFF_FFFF	NSC
External memories	0x6000_0000 0xDFFF_FFFF	Non-secure

1. NSC = non-secure callable.

The default state of memory is secure when SAU is disabled. This is the state in which the core starts at reset, meaning the core can't switch to non-secure world. If both IDAU and SAU are present in a system, the attribution from SAU is used

If both IDAU and SAU are present, the attribution from SAU is used unless IDAU provides a higher attribution for an address.

Hence, IDAU is used to define a default, unchangeable memory layout and let the developer make the final attribution in SAU. Programming of SAU happens only in secure world and will be one of the first action processed at the beginning of the program. The program must logically start from the NSC region.

The table below shows the logic of determining the security attribution for a given address. To determine the security attribution of an address, the core presents the address of the SAU and IDAU, which in turn signal to the core the security attributed of the address:

Jean-Jacques Meneu

IDAU security attribution	SAU security attribution	Final security attribution
non-secure	secure	secure
	NSC	NSC
	non-secure	non-secure
NSC	secure	secure
	NSC	NSC
	non-secure	**NSC**
secure	secure	secure
	non-secure	**secure**
	non-secure callable	**secure**

Note that the STM32L552 does not define secure attribute for the IDAU. As the SAU is present, it is the programming of the SAU that will make some regions completely secure.

The SAU is part of the core peripheral register regions. As it is part of the code, the details are given in the programming manual.

Table 47. Core peripheral register regions

Address	Core peripheral	Description
0xE000E000-0xE000E00F	System control and ID registers	Includes the Interrupt Controller Type and Auxiliary Control registers
0xE000ED00-0xE000ED8F		Section 4.3 System Control Block
0xE000EDF0-0xE000EEFF		Debug registers in the SCS
0xE000EF00-0xE000EF8F		Includes the SW Trigger Interrupt Register
0xE000E010-0xE000E0FF	System timer	Section 4.4 System timer, SysTick
0xE000E100-0xE000ECFF	Nested Vectored Interrupt Controller registers	Section 4.2 Nested Vectored Interrupt Controller
0xE000ED00-0xE000EDEF	Security Attribution Unit	Section 4.5.1 Security Attribution Unit -
0xE000ED90-0xE000EDB8	Memory Protection Unit	Section 4.5.9 Memory Protection Unit
0xE000EF30-0xE000EF44	Floating-Point Unit	Section 4.6 Floating-Point Unit

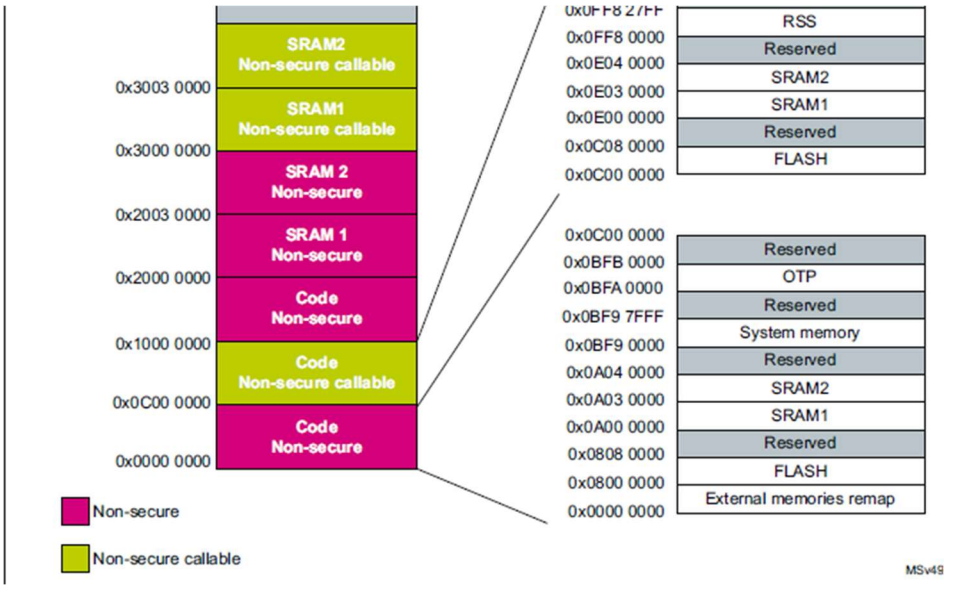
The SAU is internal to the core and is configurable at the address 0x0E00ED00

- Only privileged accesses to the SAU registers are permitted
- The SAU registers are RAZ/WI (Read as Zero, Write Ignored) when accessed from Non-secure state
- The SAU registers are not banked between Security states

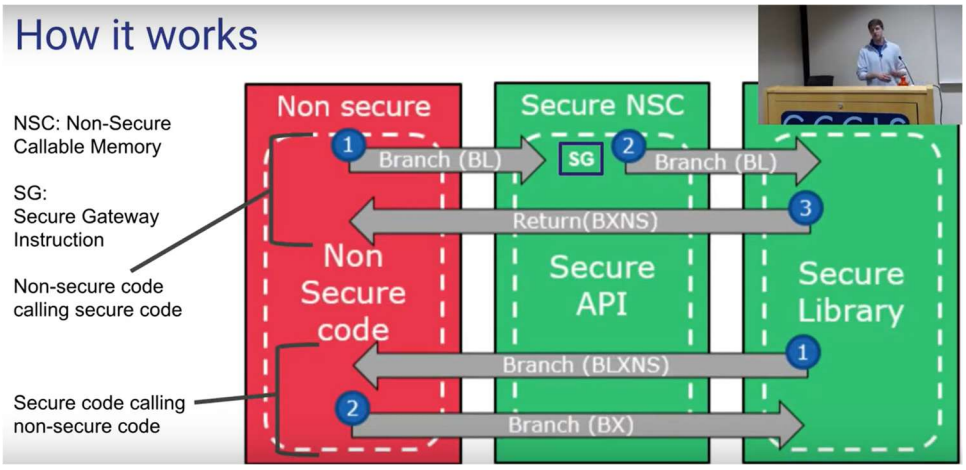
Jean-Jacques Meneu

The SAU is software configurable and physically located within the CPU core; its implementation is defined by ARM. The IDAU is defined by a functional interface, the implementation being up to the silicon manufacturer. On each CPU memory transaction, both the SAU and IDAU perform an address lookup, and respond with the security attribute associated with the memory address. If an address maps to regions
defined by both, their combined response will select the highest security level.

By default, memories are partitioned between non-secure et non-secure callable regions. For instance, The STM32L552 integrated 512KB is Flash. The non-secure goes from 0x0800 0000 up to 0x0808 0000. The non-secure callable starts at 0x0C00 0000. This is an alias and the same offset cannot be at the same time on-secure and non-secure callable. In other words, it is the same memory cells that can be accessed by 2 difference ways.



In order to jump from the non secure word to the secure world, the program must go through the Non-secure callable section that starts by an SG instruction.



Jean-Jacques Meneu

These steps can be verified thanks to the debugger. As said, to jump to a function located in a secure section, 3 steps occur:

1. Jump to a veneer located at the end of program in the non-secure section

```
..... ..... ...
                __Secure_Toggle_Pin_veneer:
08041b30: 5f f8 00 f0   ldr.w   pc, [pc]        ; 0x8041b34 <__Secure_To
08041b34: 11 e0         b.n     0x8041b5a <MSIRangeTable-66844386>
08041b36: 03 0c         lsrs    r3, r0, #16
08041b38: 00 00         movs    r0, r0
08041b3a: 00 00         movs    r0, r0
```

2. In the above example, pc value becomes 0x0C03E011. Then, the program jumps to the SG instruction in the Non-Secure Callable section

```
..... ..... ...
                Secure_Toggle_Pin:
0c03e010: 7f e9 7f e9   sg
0c03e014: c2 f7 a2 ba   b.w     0xc00055c <Secure_Toggle_Pin>
0c03e018: 00 00         movs    r0, r0
```

3. The SG function is mandatory as the first instruction. The ARM documentation explains this instruction

C2.4.144 SG

Secure Gateway marks a valid branch target for branches from Non-secure code that call Secure code.

This instruction sets the Security state to Secure if its address is in Secure memory. If the address of this instruction is in Non-secure memory, it behaves as a NOP.

If the PE was previously in Non-secure state:

- This instruction sets bit[0] of LR to 0, to indicate that the return address will cause a transition from Secure to Non-secure state.
- If the Floating-point Extension is implemented, this instruction marks Secure floating-point state as inactive, by setting CONTROL_S.SFPA to 0. This indicates that the floating-point registers do not contain active state that belongs to the Secure state.

If the Security Extension is not implemented, this instruction behaves as a NOP.

Note

SG is an unconditional instruction and executes as such both inside and outside an IT instruction block.

4. In the end, the function located in the secure section is called

```
0c00055b: 30 80         adds    r0, #0
69                      CMSE_NS_ENTRY void Secure_Toggle_Pin(void) {
                        Secure_Toggle_Pin:
0c00055c: 80 b5         push    {r7, lr}
0c00055e: 00 af         add     r7, sp, #0
70                        HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_7);
0c000560: 80 21         movs    r1, #128        ; 0x80
0c000562: 1d 48         ldr     r0, [pc, #116]  ; (0xc0005d8 <Secure_
```

5. With the linker file, it can be verified the consistency with the assembly code

```
/* Memories definition */
MEMORY
{
  RAM    (xrw)   : ORIGIN = 0x30000000,  LENGTH = 96K    /* Memory is divided. Actual start is 0x30000000 and actual length is 192K */
  RAM2   (xrw)   : ORIGIN = 0x30030000,  LENGTH = 64K
  FLASH  (rx)    : ORIGIN = 0x0C000000,  LENGTH = 248K   /* Memory is divided. Actual start is 0x0C000000 and actual length is 512K */
  FLASH_NSC (rx): ORIGIN = 0x0C03E000,  LENGTH = 8K     /* Non-Secure Call-able region */
}
```

In order to call functions from the secure functions or non-secure functions, 2 attributes must be used that are explained below:

6.3.1. Defining Non-secure Entry Functions

An *non-secure entry* function in this context refers to a function defined in secure memory that can be called from non-secure code via a secure gateway veneer. Such a function must be annotated with a *cmse_nonsecure_entry* attribute.

6.3.2. Calling Non-secure Functions

A function defined in non-secure memory may only be called from code in secure memory via a function pointer that has been annotated with a *cmse_nonsecure_call* attribute. Such a function pointer is referred to as a *non-secure function pointer*.

Deliverables

- Follow STM32 MOOC security Part 3- STM32 Security Features 19 – TrustZone lab to understand how to configure an application with TrustZone enable.
  Warning: LED is on PC7 (different of the MED presented in the video)

https://www.youtube.com/watch?v=AoE2j7gKZHw&list=PLnMKNibPkDnFzux3PHKUEi14ftDn9Cbm7&index=19

- Write a function in the secure area that transmit a message in UART each time the push button is pushed.
- Declare a static variable to count the number of times the button is pushed and send this information via uart
- Check the location address of the static variable to confirm understanding of isolation